AI Compiler Performance Engineering

Auto-Loops Books — ch29

Contents

1 YiRage Runtime Layer and Persistent Kernel Execution **1**

1.1 yirage stack layers 4

1.2 persistent kernel decode 8

1.3 superoptimize handoff 13

1.4 hw registry runtime 17

1.5 deps build native 20

1.6 YiRage runtime decode benchmarking methodology 24

1.7 Key Takeaways 31

1.8 Conclusion 31

Bibliography **33**

Chapter 1

YiRage Runtime Layer and Persistent Kernel Execution

Chapter 28 placed inference *frameworks* above the decode step; this chapter covers YiRage *below* exported graphs—the C++ runtime, Persistent Kernel launcher, and backend registry that turn `superoptimize()` results into repeatable decode cells (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Source tree: git submodule `deps/YiRage` (upstream (Chen and YiRage Contributors, 2026)); init via `git submodule update -init -recursive deps/YiRage` (see `deps/README.md`).

TaxBreak counts launches/token inside the decode box; Memory-Floor counts bytes/token and graph replay relief (Vellaisamy *et al.*, 2026; Chen, 2026). Compiler passes from Part VI–VII emit fused plans; **Layer 5 runtime** owns device memory discipline, launch templates, and backend selection so those plans survive fleet load (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Without runtime contracts, `superoptimize()` wins evaporate at the framework plugin boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Readers should map each Layer 5 API to Chapter 1 counters: `PersistentKernel` collapses launches/token; runtime residency metadata cuts bytes/token; `HardwareRegistry` constrains search to the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a;

SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Chapter 30 stitches framework schedulers to this runtime; here we stay inside `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at runtime means treating device memory like registers with lifetimes: allocations inside the decode step are bugs unless proven on-chip (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Layer 5 code paths should log residency class transitions the way Chapter 6 planners annotate IR (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). When runtime logs show unexpected HBM allocs per token, return to Layer 3 fusion legality before tuning scheduler batch windows (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

This chapter is the mirror of Chapter 28: frameworks own everything outside the decode box; runtime owns everything inside once the framework plugin invokes PK (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Integration meetings should draw that box first, then assign Layer 1–5 owners (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Runtime engineering begins where compiler IR ends. The fused graph is no longer a data structure in Python; it is a device resident program with allocation pools, synchronization points, and launch templates that must survive millions of decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams new to YiRage often stop measuring after superoptimize reports a winning tile config. Production goodput is decided in Layer 5 when paging pointers arrive from vLLM, when CUDA graphs replay, and when softmax carriers stay in registers instead of spilling to HBM (Kwon *et al.*, 2023; Chen, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016). Treat runtime as a first class performance surface with its own counters, owners, and release train (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025;

Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

The submodule layout under deps mirrors this responsibility. Python examples in the book intentionally point at pinned upstream APIs, but Layer 5 behavior lives in C++ under libyirage runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When documentation disagrees with profiler traces, trust traces and file issues against runtime launch paths rather than re tuning scheduler batch sizes alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Chapter 30 will show how framework schedulers call into this stack; here we keep the camera inside Layer 5 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Three questions gate every runtime review. First, does warmed PersistentKernel beat eager launches/token on the fleet SKU at batch one (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Second, does bytes/token beat the pre fusion baseline at context 512, 2048, and 8192 (Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Third, does integration preserve paging ABI and softmax carriers through ten thousand cancel and resume cycles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). If any answer is no, stop feature work and return to superoptimize or descriptor design (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime tuning knobs cannot fix illegal fusion or missing manifest fields (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Runtime ownership should appear on the org chart, not only in architecture slides. Layer five engineers maintain launch descriptor schemas, pool allocator invariants, and graph capture bucket tables that framework teams consume as stable ABI. When paging block size changes in vLLM, runtime must publish a semver bump before framework plugins ship; when superoptimize emits a new softmax carrier layout, runtime validates manifests at load time rather than discovering divergence during golden token tests. On-call runbooks should list the three counter worksheets from Chapter one beside PK warmup steps so incidents do not devolve into guessing whether regression lives in search, runtime, or scheduler batching.

Production fleets rarely fail because a single kernel is slow; they fail because two replicas run different artifact digests, because cancel paths leak pool slots

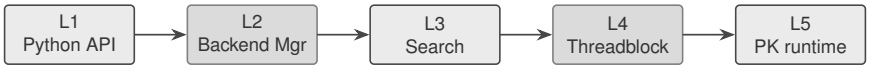


Figure 1.1: YiRage five-layer stack: Layers 1–4 search and lower; Layer 5 Persistent Kernel runtime executes decode cells on device.

under continuous batching, or because a canary node reports the wrong chip id to the registry. Layer five observability therefore emphasizes per-step JSON logs with fingerprint, registry id, descriptor version, launches count, and byte estimate aggregated into dashboards that framework SREs can read without reading C++. Postmortems should replay ten thousand decode steps from a single request id and verify pool high-water marks return to baseline after cancel. Treat runtime regressions as release train defects: roll back the artifact bundle, not only the framework container image.

1.1 yirage stack layers

YiRage stack layers follow the five-layer model in Table 1.1: Python API → Backend Manager → Search/Strategy → Threadblock ops → **Persistent Kernel runtime** (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Parts IV–VI focused on compiler passes and IR legality; Part VIII completes the stack with execution and device memory management (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Chen, 2020).

Layer boundaries and debugging. Production incidents often misattribute Layer 3 search regressions to Layer 5 runtime or vice versa (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When bytes/token regress, diff the fused μGraph fingerprint first (Layer 3 output); when launches/token regress, inspect PK launch descriptors and graph capture buckets (Layer 5) (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Layer 2 must reject cross-backend artifacts silently loaded from cache—a Hopper-tuned plan must not execute on XDNA without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Industrial teams should staff Layer 5 with engineers who read nsys and who

Table 1.1: YiRage layer responsibilities and decode goodput levers.

Layer	Component	Primary lever on decode
L1	Python `graph`, `PersistentKernel`	User-facing API; mode and backend selection (Chen and YiRage Contributors, 2026; Chen, 2020)
L2	Backend Manager	Routes CUDA/CPU/NPU backends; same-backend enforcement (Chen and YiRage Contributors, 2026; AMD, 2024b,a)
L3	Search / `superoptimize`	Fusion plans, tile configs, fingerprint cache (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026)
L4	Threadblock ops	Lowered primitives before PK packaging (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024)
L5	PK runtime	Launch collapse, device memory pools, graph replay hooks (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026; Davies *et al.*, 2025)

understand paging ABIs from Chapter 28 (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler teams own Layers 1–4 artifacts; runtime teams own launch stability, pool lifetimes, and graph replay (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When both groups share one on-call rotation, counter worksheets prevent thrash: if launches/token improved but bytes/token worsened, rollback Layer 3; if both stalled, inspect descriptor versions at the framework boundary (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 logs should include enough context to reproduce PK configs without re-running full search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Threadblock lowering is the last human-readable checkpoint before opaque device code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When numerics diverge, compare threadblock dumps against eager reference at the same paging layout (Kwon *et al.*,

2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Do not tune softmax epsilons in runtime unless threadblock IR confirms carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This discipline keeps runtime hotfixes from masking compiler legality bugs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Python API surface. Layer 1 exposes `yirage as yr`, `yr.new_graph()`, tensor helpers, and `graph.superoptimize(backend=...)` entry points documented in upstream `docs/API_REFERENCE.md` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Application code should treat graphs as immutable after superoptimize: runtime consumes frozen configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Mixing eager edits post-search breaks fingerprint caches and framework plugin contracts from Chapter 28 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Backend Manager routing. Layer 2 selects installed backends (CUDA, CPU, NPU) and validates build flags from native extensions (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Import failures here surface as missing `yirage.core` or `libyirage_runtime`—not as silent CPU fallbacks in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). CI should compile each target backend separately; triplet tests compare counters, not import success alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Search versus runtime split. Layer 3 explores fusion and tiling; Layer 5 assumes a fixed plan (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Autotune during serving is an anti-pattern—it destroys graph capture stability and p99 latency (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Offline superoptimize plus pinned artifacts is the production pattern (Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Layer 4 threadblock ops are the last IR-like surface before opaque device binaries—debug dumps here when PK numerics diverge from eager reference (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Observability per layer. Layer 1 logs Python call sites; Layer 5 emits device timestamps and byte counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Correlate layer logs with framework traces from Chapter 28 using shared request ids (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Without correlation, runtime teams optimize kernels while queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Export a single JSON line per decode step containing layer id, registry id, artifact fingerprint, launches count, and byte estimate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Aggregates belong in fleet dashboards; per step JSON enables postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teach on-call engineers to filter on fingerprint first, registry id second, and framework version third (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Most regressions are mismatched artifacts, not mysterious hardware noise (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Layer handoffs deserve explicit interface tests in CI, not tribal knowledge. After superoptimize completes, assert that Layer four threadblock dumps include every carrier field the Layer five loader expects. After PersistentKernel construction, assert that warmup completes within a bounded step count and that steady decode reuses the same pool handles across one thousand tokens. When Layer two rejects a backend, the error message should name the missing native extension so developers do not confuse import success with PK readiness. Stack diagrams in documentation are useful, but automated boundary tests prevent silent drift when upstream YiRage refactors internal module names while preserving Python API

spelling.

Multi-hardware delta. Layer 4–5 differ sharply: CUDA PK uses graph capture and TMA-aware pools; XDNA uses static AIE binaries; CPU uses vectorized host loops (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Same Python API must not imply identical launch semantics (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 1.1.1. Search on laptop CPU then deploy CUDA PK without re-superoptimize: numerics may match while launches/token and bytes/token diverge—violates same-backend rule (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Worked layer walkthrough. Start from a minimal decode graph in Layer 1 and execute superoptimize on the fleet GPU (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Inspect Layer 3 logs for chosen fusion boundaries and tile sizes, then dump Layer 4 threadblock lowering for the attention seam (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Instantiate PersistentKernel in decode mode and run ten tokens at context 512, 2048, and 8192 (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Record launches per token, HBM bytes per token, and wall time per token separately for each bucket (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Only after Layer 5 counters flatten should you integrate the framework plugin from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Skipping the isolated PK microbench hides descriptor bugs that become production incidents under continuous batching (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

1.2 persistent kernel decode

Persistent kernel decode uses `yr.PersistentKernel(mode="decode", backend=..., fallback_backends=[...])` to select backend execution with graceful CPU/NPU

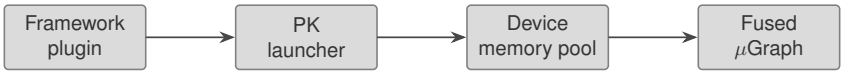


Figure 1.2: Persistent kernel decode path: framework passes slot and page metadata; PK runtime launches fused decode cells with collapsed host overhead.

fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022). The `persistentkernel` object wraps a long-lived device program: internal loops over decoder layers replace per-op host launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). `mode="decode"` selects shape buckets and numerics carriers tuned for BS=1 token steps rather than prefill-only tiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Same-backend rule. Search and execute on the installed target—do not profile on CPU and ship CUDA kernels without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet labels (GPU SKU, driver, CUDA version) must match `HardwareRegistry` entries used during `superoptimize` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Warmup passes prime PK device memory pools and graph capture—cold first token latency is a runtime concern distinct from compiler fusion quality (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Decode PK differs from training kernels in occupancy goals (Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training chases large batch matmul utilization; decode chases minimal bytes per token at batch one (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Persistent loops amortize layer weights across tokens inside one launch, but only if softmax and norm carriers remain on chip between layers (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile carrier lifetimes explicitly when PK shows

unexpected HBM traffic (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Cancellation and mid-sequence abort must release PK pool slots without leaking device memory (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Frameworks free paging blocks on cancel; runtime must mirror that lifecycle in pool accounting (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Long-running servers accumulate leaks if cancel paths skip pool teardown (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Stress tests should randomize cancel timing under continuous batching before fleet promotion (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Fallback backends. `fallback_backends` enable dev laptops and CI smoke tests without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Production must pin primary backend to fleet SKU; fallbacks are for correctness checks, not performance baselines (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Accidentally serving through CPU fallback while GPU PK is available reads as mysterious fleet cost overrun (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Device memory pools. Layer 5 allocates persistent buffers for weights, KV views, and softmax carriers across decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework paging passes block table pointers into PK launch descriptors—runtime must not copy KV to contiguous staging each token (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies show up as bytes/token regressions with unchanged fusion plan (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Launch descriptor contract. Each decode step receives slot mask, page table

handle, step id, and sampling flags from the framework plugin (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PK runtime validates descriptor version before launch—version skew must hard-fail, not produce wrong tokens (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Golden-token tests compare eager reference against PK for each descriptor version bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Prefill versus decode PK modes. `mode="decode"` selects persistent loops tuned for single-token advance; prefill may use separate superoptimize profiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mixing prefill tiles in decode PK wastes bandwidth—tag modes explicitly in artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Framework schedulers call the correct PK mode per engine iteration (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Shape buckets for decode should align with framework max batch width and max context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Each bucket needs a warmed PK instance and a graph capture template (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing buckets fall back to slow paths that show up as tail latency spikes rather than mean regression (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document bucket coverage in runbooks beside paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Persistent kernel tuning is a memory lifetime problem disguised as a launch problem. Each decoder layer expects weight tensors, norm statistics, and attention softmax carriers to arrive at fixed addresses across steps so the device loop can branch minimally. When runtime allocates fresh staging buffers per token, bytes per token rises even though TaxBreak launch counts look excellent. Profile with

memory transaction counters and verify that page table pointers from the framework pass straight into gather kernels without intermediate copies. When carriers spill to high bandwidth memory between layers, return to Layer three fusion legality before widening graph capture buckets.

Stress decode under adversarial scheduler patterns: alternating prefill and decode on the same slot, random cancels at ten percent of steps, and context growth from five hundred twelve to eight thousand one hundred ninety two within one session. PK pools must survive these patterns without monotonic growth in reserved bytes. Framework plugins should not assume PK warmup happens once per process lifetime if workers restart frequently in Kubernetes; document whether warmup is idempotent and how long it blocks readiness probes. Serving teams often tune liveness thresholds without accounting for first capture latency, then blame runtime for pod churn that is actually missing warmup in the probe design.

Multi-hardware delta. CUDA PK pairs with CUDA Graph buckets; XDNA PK loads AIE binaries with static DMA chains; CPU PK uses OpenMP-blocked loops (NVIDIA, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Identical Python `persistentkernel` constructor arguments may map to different launch paths (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 1.2.1. Decode cell (BS=1, ctx=2048): PK launch drops launches/token from TaxBreak hundreds to single digits while framework paging unchanged—runtime owns launch collapse, not batching (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; NVIDIA, 2024; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

PK versus CUDA graphs alone. Memory-Floor style graph replay removes host launch storms but cannot fix illegal fusion (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). PersistentKernel combines graph stability with compiler chosen fusion inside the replayed region (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams seeing graph wins without byte wins should escalate to superoptimize rather than adding more graph buckets (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*,

2023; Goodfellow *et al.*, 2016). PK is not a replacement for paging or batching; it replaces per op launches inside the decode cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

MoE and expert parallel hooks. Large MoE models add expert parallel communication below the PK entry point (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Open DeepEP components illustrate all to all patterns that must compose with PK memory pools rather than allocate per token (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile EP separately from PK fusion when bytes/token regress on MoE checkpoints (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

1.3 superoptimize handoff

Superoptimize handoff bridges `graph.superoptimize(backend=...)` outputs to runtime launch: fused μGraphs, fingerprint verification, and cached kernel configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 search emits a serialized plan; Layer 5 loads it into PK launch templates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework hooks (Chapter 28) must not strip metadata required for capture or MegaKernel legality (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Handoff is where many projects stall: search finishes, a demo looks fast, but no artifact pipeline reaches serving (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Treat superoptimize output like a container image: digest, manifest, signature, and rollback tag (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins should load by digest at process start, not by mutable local paths (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Canary deploys swap digest on

a fraction of replicas while comparing counter dashboards (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Without digest discipline, two replicas silently run different PK programs under the same endpoint (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fingerprints and cache keys. Superoptimize caches results keyed by IR fingerprint plus backend id and hardware registry hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework upgrades that change page-table ABI must bump cache salts or risk loading incompatible PK binaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). CI should verify fingerprint mismatch fails closed—never silently fall back to unfused eager (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Graph to kernel packaging. Threadblock lowering produces device-specific binaries or CUDA C++ sources consumed by PK runtime (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Handoff artifacts include tile sizes, double-buffer depth, and softmax carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing carrier metadata passes unit tests on square tensors and fails on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

Warmup and steady state. First PK launch may compile, allocate pools, and capture graphs; steady decode reuses them (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet benchmarks must separate warmup from sustained tokens/s (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production SLOs often ignore first-token compile; decode p99 must use warmed PK state (Davies *et al.*, 2025;

Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Artifact storage layout. Store superoptimize outputs as versioned bundles: fingerprint, registry id, backend, blob, and manifest JSON (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework plugins fetch bundles by hash at startup—avoid embedding blobs inside Python wheels without hash checks (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Rollback is redeploy prior bundle, not revert framework scheduler alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Legality metadata handoff. Export softmax carriers, page-table layouts, and graph-capture flags as manifest fields (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Runtime refuses launch if manifest missing required fields—fail closed (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This prevents partial exports from Chapter 28 plugins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Manifest evolution requires semver: breaking carrier layout bumps major version; additive fields bump minor (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins declare supported manifest major before loading artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Mixed major versions across replicas is a deployment error, not a runtime tuning opportunity (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 30 runbooks formalize this three party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Artifact promotion should resemble a binary signing pipeline. Search cluster produces a bundle; staging loads it read only; golden token harness compares hashes against eager reference on three context buckets; production receives a digest pin in the framework config. Never promote because a demo video looked

fast on one laptop. Store the promotion checklist JSON beside the bundle so auditors can reconstruct who signed off on launches per token and bytes per token deltas. When a framework upgrade changes only scheduler policy but not paging ABI, artifacts may carry forward; when block table layout changes, invalidate caches even if IR fingerprint unchanged. Handoff meetings should read manifest fields aloud until boring repetition makes missing fields impossible.

Offline search and online runtime must share a single hardware registry revision checked into version control. Drift between search cluster registry and serving cluster labels produces plans that are legal yet slow, which is harder to detect than hard failures because tokens still look plausible. Automate a nightly job that compares detect current chip output on every node class against expected registry ids and pages when mismatches exceed zero. Superoptimize caches keyed only on IR miss these failures until bytes per token creep upward over weeks.

Multi-hardware delta. Cache entries are not portable across backends—re-superoptimize per target even when IR matches (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 1.3.1. Fingerprint hit loads cached PK plan; framework block size change without re-search yields wrong KV gathers—cache must invalidate on paging ABI change (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Handoff checklist before fleet deploy. Verify manifest lists backend id, registry id, fingerprint, paging ABI version, and softmax carrier schema (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Run golden tokens on three context buckets with warmed PK (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Compare launches/token and bytes/token against prior artifact in the same dashboard row (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Sign off only when both counters move or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Store the checklist JSON beside

the artifact bundle for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

1.4 hw registry runtime

Hw registry runtime (`HardwareRegistry`, `detect_current_chip()`) supplies chip models for search configs—Hopper vs XDNA vs M3 Max tile candidates (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). The `hardwareregistry` maps detected devices to search spaces: SM count, TMA availability, L2 size, NPU tile SRAM, CPU vector width (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). `detect_current_chip()` runs at import or first superoptimize—runtime detection must align with fleet labels used in framework schedulers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a,b; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Search space pruning. Registry entries prune illegal tile shapes before autotune explosion (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Hopper profiles enable TMA double-buffer candidates; XDNA profiles restrict to static DMA chains (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Wrong chip detection yields legal-but-slow plans—profile bytes/token on representative ctx buckets (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry entries should be versioned documents checked into the same repo as artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When hardware vendors ship new SKUs, add entries before marketing promises fleet migration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Each entry lists peak bandwidth, on-chip capacities, and forbidden fusion patterns for

that target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Search reads these as hard constraints, not hints (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Detect current chip at process start and log the resolved registry id in every performance trace (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Post-incident review should filter traces by registry id to spot partial rollouts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mixed ids behind one load balancer explain bimodal latency histograms (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry maintenance is a hardware catalog discipline, not a one time bootstrap script. Each entry documents peak bandwidth, on chip SRAM, vector width, tensor engine availability, and forbidden fusion patterns for that SKU. When vendors ship new accelerators, add entries before marketing announces fleet migration dates. Search reads entries as hard constraints; runtime logs the resolved id on every decode step so capacity planners can correlate artifact lines with instance types. Retire entries only after all pinned artifacts migrate and dashboards show zero traffic on the old id for two release cycles.

Virtualized and partitioned GPUs distort detection unless runbooks override generic ids with fleet labels from node metadata. Multi instance GPU slices may report parent SKU names that mislead tile search toward shapes that fit the parent but not the slice memory budget. Time shared virtual machines may hide NPU presence entirely until first superoptimize call. Document override procedures and require platform teams to validate overrides quarterly against vendor tools output.

Fleet inventory sync. Kubernetes node labels, cloud instance types, and driver versions should map to registry ids (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mismatch between scheduler placement and PK chip model causes silent suboptimal tiles (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin superoptimize artifacts per registry id in artifact stores (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Multi-SKU rollouts. Rolling PK updates across heterogeneous fleets requires

parallel artifact lines per chip entry (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search even when CUDA version matches (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry-driven autotune budgets. Cap search trials per chip model to keep CI bounded (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Registry metadata feeds roofline estimates that prune hopeless tile shapes before device trials (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Log pruned candidates for audit when bytes/token underperform expectations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Virtualization pitfalls. Containers sharing GPUs must expose consistent chip ids to detect_current_chip (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MIG slices and time-sliced VMs may report generic ids—override with fleet labels when needed (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Multi-hardware delta. `detect_current_chip` on VM passthrough GPU may report wrong SKU—validate against `nvidia-smi` product name in runbooks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 1.4.1. Fleet moves from H100 to H200: rerun superoptimize with updated registry entry; reuse old fingerprint cache caused 8% bytes/token regression until invalidated (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Registry operations runbook. Onboard a new GPU SKU by adding a registry entry, running a bounded superoptimize search, and publishing artifact bundles tagged with the new id (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Update Kubernetes node labels to match registry ids before shifting traffic (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). During incidents, compare detect_current_chip output with node labels before blaming framework schedulers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Retire old entries only after all pinned artifacts migrate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

1.5 deps build native

Deps build native: `import yirage` requires `yirage.core` + `libyirage_runtime`—no Python-only native build path (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule init: `git submodule update -init -recursive deps/YiRage`; build via `YIRAGE_BACKEND=cuda pip install -e deps/YiRage` (see upstream `docs/INSTALLATION.md`) (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Native extension layout. Python package wraps C++ runtime under `deps/YiRage/yirage/` with compiled `yirage.core` module linking `libyirage_runtime` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Missing .so files import as pure Python stubs that fail at first PK call—smoke tests must invoke `PersistentKernel`, not only `import yirage` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Building from source ties book examples to reproducible hashes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Pin submodule commit in release notes whenever prose references API symbols (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Nested submodules may require recursive init; document network and disk expectations in deps README (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Wheel-based installs are convenient for demos but hide compiler flags (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production should prefer editable installs from pinned submodule with recorded nvcc and driver versions (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When wheels and submodule builds diverge, counter worksheets become incomparable across environments (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Backend build flags. `YIRAGE_BACKEND=cuda|cpu|npu` selects compile targets; nested submodules may pull vendor SDKs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Docker images for serving should bake the same backend flags as search CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Drift between build and runtime containers is a common PK load failure (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Version pin with book repo. Pin submodule SHA in `.gitmodules` or recorded commit; Chapter 30 runbooks require matching `deps/YiRage` revision with framework plugin ABI (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Book examples reference APIs at pinned SHA—update prose when bumping submodule (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Developer workflow. Typical loop: edit IR or search config → `superoptimize` on target SKU → run PK decode microbench → export artifact hash for framework plugin (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Skipping native rebuild after C++ runtime change produces stale .so mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

CI smoke template. Minimal pipeline: submodule init, backend-specific compile, import yirage, superoptimize tiny graph, PersistentKernel decode ten tokens, assert golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend with BS=1 counter worksheet from Chapter 1 before promoting artifacts to staging (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Docker and reproducibility. Lock CUDA driver, toolkit, and YiRage SHA in serving images (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Non-reproducible PK builds frustrate bisect when fleet regressions appear (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Record compiler flags in artifact manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Serving images should compile YiRage once in the build stage and copy artifacts into the runtime stage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime stage installs matching driver user space libraries only; never compile on production pods (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Init containers can verify artifact digest against manifest before accepting traffic (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). This pattern prevents silent drift between search cluster and serving cluster (Chen and YiRage Contributors, 2026; Chen, 2020; Davies

et al., 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Native build failures often masquerade as Python packaging issues. Missing nvcc, wrong driver user space libraries, or stale build directories produce import success followed by opaque failures at first PersistentKernel call. CI must execute the full smoke template on the same backend flag as production containers, including ten token decode and golden hash comparison. Developers on laptops without GPUs may compile CPU backend for unit tests but must never promote CPU searched artifacts to CUDA serving. Record compiler flags, submodule SHA, and backend enum in artifact manifests so bisect can distinguish runtime code changes from toolchain drift.

Book readers reproducing examples should init the submodule recursively, install editable with explicit backend flag, and run PK decode microbench before reporting API mismatches. Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage with submodule bump and manifest semver bump in this repository. When documentation and profiler traces disagree, trust traces and file issues against runtime launch paths.

Multi-hardware delta. CUDA builds need compatible nvcc and driver; XDNA builds need Ryzen AI SDK paths documented upstream (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). CI matrices should include at least one compile plus PK smoke per supported backend (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 1.5.1. `pip install -e deps/YiRage` on dev laptop without CUDA toolkit: import succeeds, PK decode fails at runtime—gate merges on PK smoke, not import (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Local dev versus fleet parity. Developers may compile CPU backend locally for unit tests while fleet runs CUDA PK (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016). Never promote CPU searched artifacts to GPU serving (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Provide a small GPU CI runner that executes the same smoke template as production (Chen and YiRage Contributors, 2026; Chen, 2020;

Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document driver and toolkit versions in deps README when bumping submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Book readers should clone submodule recursively before reporting API mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

1.6 YiRage runtime decode benchmarking methodology

Benchmarking Layer 5 decode extends Chapter 1 counters with runtime-specific ledgers: PK warmup steps, pool high-water marks, graph capture bucket hits, and artifact fingerprint stability across ctx buckets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx $\{512, 2048, 8192\}$, pinned `deps/YiRage` SHA, matching registry id, and framework paging ABI from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up must include PersistentKernel construction, first graph capture, pool allocator steady state, and at least one thousand steady decode steps—cold import success alone is not a valid production benchmark (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report first-token latency and token-k steady ms/token separately; product SLOs often bind to the latter while UX marketing highlights the former (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet runs should compare eager framework decode, PK-only microbench, and full framework plugin path on the same SKU (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A PK win that disappears after plugin integration signals descriptor ABI mismatch, not

Metric	Layer 5 decode meaning
Launches/token	PK + graph replay vs eager baseline
Bytes/token	Pool residency + paging traffic
Pool high-water	Leaks under cancel/resume
Fingerprint match	Artifact drift across replicas
Registry id	SKU-aligned search vs runtime
Graph bucket hit rate	Capture stability under ctx growth

Table 1.2: Runtime decode counter worksheet (extends Chapter 1 TaxBreak / Memory-Floor).

"framework overhead" (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 1.6.1. Runtime decode gate: warmed PK at ctx=2048 must show launches/token within 5% of search logs *and* bytes/token within 10% of Memory-Floor worksheet; attach fingerprint, registry id, and ten-step pool ledger (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cancel and resume stress. Continuous batching injects cancel paths that microbenches skip (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark ten thousand decode steps with randomized cancel/resume at 1% rate; pool high-water must return to baseline within bounded slots (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Fail CI when cancel paths allocate fresh HBM per step (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Artifact pinning in CI. Nightly jobs must load the same artifact bundle as staging: fingerprint, manifest semver, compiler flags, and submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Drift between search cluster artifacts and serving replicas explains bimodal latency histograms better than kernel tuning (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Publish golden hashes and pool ledgers beside every Layer 5 release note in CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Framework boundary replay. After PK microbench passes, replay the same counters through the Chapter 28 plugin with identical paging block size and request ids (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Deltas here isolate scheduler tax from runtime tax (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never promote PK artifacts to staging or production without this second worksheet (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

Multi-SKU fleet sweeps. Heterogeneous fleets need parallel benchmark matrices keyed by registry id—reuse Hopper numbers on Blackwell only after re-search and re-warmup (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Dashboards should slice launches/token and bytes/token by registry id, not aggregate fleet averages (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon

et al., 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Operator documentation. Ship runtime benchmark artifacts with every promotion: ctx buckets, warmup protocol, pool ledger, graph bucket table, and triplet comparison summary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, on-call engineers cannot distinguish artifact drift from framework queueing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging a Layer 5 release, require green PK smoke, cancel stress, plugin replay, and fingerprint match across three ctx lengths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partial green is how production inherits week-long triage across runtime and framework teams (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Emit structured logs at PK load, warmup complete, steady decode entry, and graph capture miss events (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip verbose fields but retain stable event names so SRE dashboards correlate runtime regressions with artifact promotions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Extend runtime benchmarks to ctx=16384 when product SLAs allow paging beyond 8K (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Graph capture buckets tuned at 2K often miss at 32K—fail CI when bucket hit rate drops below threshold at long ctx (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode at runtime. Draft-verify stacks double PK instances; benchmark acceptance-rate sweeps with separate pool ledgers per model (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Alias bugs between draft and verify KV pools show up only under load—include them in cancel stress matrices (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat Layer 5 decode benchmarking as a shipping contract alongside native builds—without nightly PK replay and published pool ledgers, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after a runtime merge, run this sequence before blaming silicon: (1) compare artifact fingerprints across replicas; (2) verify registry id matches node labels; (3) replay pool ledger on golden PK export; (4) diff launches/token and bytes/token against last green worksheet; (5) only then sweep driver and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping

fingerprint checks wastes days chasing framework queue noise (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Efficiency metric. Track *runtime-attributed decode efficiency*: measured tokens/s divided by roofline tokens/s after subtracting framework scheduler overhead measured at the plugin boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Runtime teams own the PK numerator; framework teams own scheduler tax in the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Native build parity in benchmarks. Benchmark jobs must compile `deps/YiRage` with the same `YIRAGE_BACKEND` flag as serving containers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CPU-backend smoke tests are valid for API drift but must never gate GPU artifact promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record nvcc, driver, and submodule SHA in every benchmark manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to Chapter 30. The runtime worksheet completes when warmed PK counters, plugin replay deltas, and manifest hashes are pinned for the production workflow in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023;

NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing any field forces Chapter 30 integration meetings to guess Layer 5 state (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression harness cadence. Check golden PK decode scripts into the book repo beside static graph exports: same tokenizer, ctx buckets, pool counters, and fingerprint assertions every nightly CI run (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Submodule bumps that change descriptor layout must fail CI before prose updates merge (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation contract. Every Layer 5 promotion note must list ctx buckets, warmup steps, steady ms/token, launches/token, bytes/token, and golden fingerprint at ctx=2048 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partners cannot replay regressions without these fields—support cycles lengthen when benchmarks stay in private spreadsheets instead of versioned artifact stores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary. Layer 5 benchmarks are complete when PK, plugin, and cancel-stress worksheets agree within published tolerances on all three ctx buckets—anything else is an integration draft, not a shippable production fleet release candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

1.7 Key Takeaways

- **The runtime is Layer 5, distinct from the compiler.** Compiler passes from Parts VI–VII emit fused plans, but the runtime layer owns device-memory discipline, launch templates, and backend selection so the plan actually executes within budget (Chen and YiRage Contributors, 2026; Chen, 2020). Treating runtime as an afterthought is how a correctly-compiled megakernel still misses p99 when the host reconstructs state every token (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Same-backend search.** Profile and ship on the target SKU; `hardwareregistry` must match fleet labels (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a).
- **Handoff is contractual.** Fingerprints, page-table ABI, and softmax carriers must survive `superoptimize` to PK launch (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026).
- **The submodule is the source of truth.** The runtime depends on the native `deps/YiRage` build, so CI must compile that submodule and smoke-test a Persistent Kernel decode rather than testing only the Python import (Chen and YiRage Contributors, 2026; Chen, 2020). An import-only test passes while the native runtime is broken, which is precisely the failure that reaches production undetected (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Counters at runtime.** Measure launches/token and bytes/token after PK warmup, not only import or search logs (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

1.8 Conclusion

YiRage runtime closes the gap between fused IR and fleet decode: Layer 5 `persistentkernel` programs collapse launches/token; device memory pools and graph hooks address bytes/token and orchestration tax (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Treat `superoptimize` artifacts, registry ids, and native builds as release-train

inputs alongside framework paging policy (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Before Chapter 30, complete one Layer 5 worksheet: warmed PK, three context buckets, launches and bytes per token logged, manifest hash pinned (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 30 integrates this layer with vLLM-class frameworks and Chapter 12 execution modes into one production workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2, 4, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more

decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 4, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32